Rotary Positional Embeddings engine

Part 2 · Communications & Protocol Officer — Rafid Mohammad Ahsan · file `02_RoPE_Positional_Engine.py` · run: `python 02_RoPE_Positional_Engine.py`

`apply_rope(x, freqs_cis)` rotates each dimension pair by m·θ_i with θ_i = 10000^(−2(i−1)/d) using complex multiplication. Verified: rotation preserves the norm; ⟨RoPE(q,m), RoPE(k,n)⟩ depends only on m − n (200 random cases). Compared with ALiBi linear penalties (distance curves and heatmaps) and analysed context extrapolation with Position Interpolation.

Run output

```
$ python 02_RoPE_Positional_Engine.py
================================================================================
USSE NOOSPHERE · SUBSYSTEM 03 · PART 2: ROTARY POSITIONAL EMBEDDINGS

=== Verification ===
norm preserved under rotation: max |‖RoPE(x)‖ − ‖x‖| = 9.54e-07
<RoPE(q,10), RoPE(k,25)> = 10.773232   <RoPE(q,50), RoPE(k,65)> = 10.773232   |diff| = 8.65e-08
relative invariance over 200 random (q, k, m, n, δ): max relative error = 1.74e-06
batched (B, h, N, d) = (2, 8, 16, 64) -> (2, 8, 16, 64)
[PASS] RoPE is a pure rotation and the score depends only on m − n.

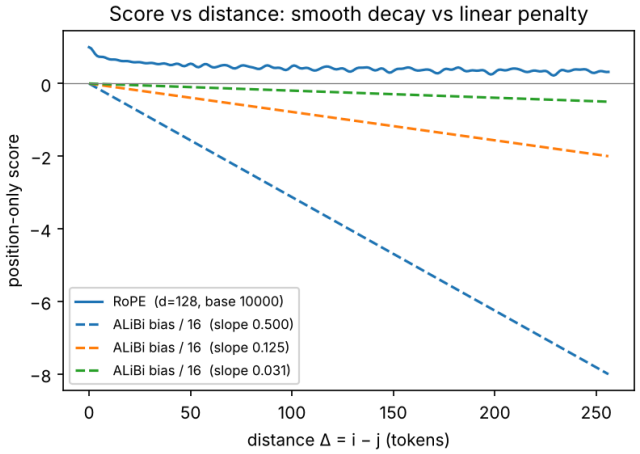
=== RoPE vs ALiBi: position-only score by distance ===
     Δ | RoPE s(Δ) | ALiBi bias (slope 0.125)
     0 |     1.000 |                  -0.00
     1 |     0.970 |                  -0.12
     4 |     0.759 |                  -0.50
    16 |     0.620 |                  -2.00
    64 |     0.477 |                  -8.00
   256 |     0.321 |                 -32.00
  1024 |     0.204 |                -128.00
  4096 |    -0.053 |                -512.00
RoPE decays smoothly and stays bounded (|s| ≤ 1); ALiBi keeps falling linearly forever,
so far tokens are pushed to ~0 weight: strong recency, simple extrapolation.

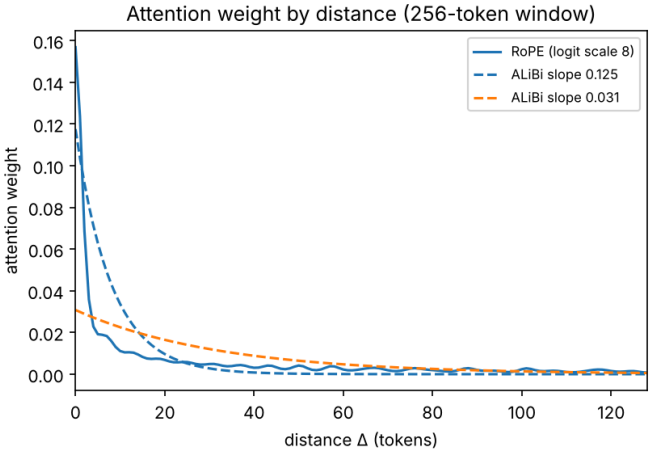
=== Context-length extrapolation (d=128, base 10000) ===
trained on 4096 tokens, asked for 32768:
  18 of 64 planes never completed a full turn in training
  slowest plane angle: 0.473 rad at train length -> 3.784 rad at target (unseen)
  Position Interpolation (scale 8×) brings it back to 0.473 rad (inside the trained range)

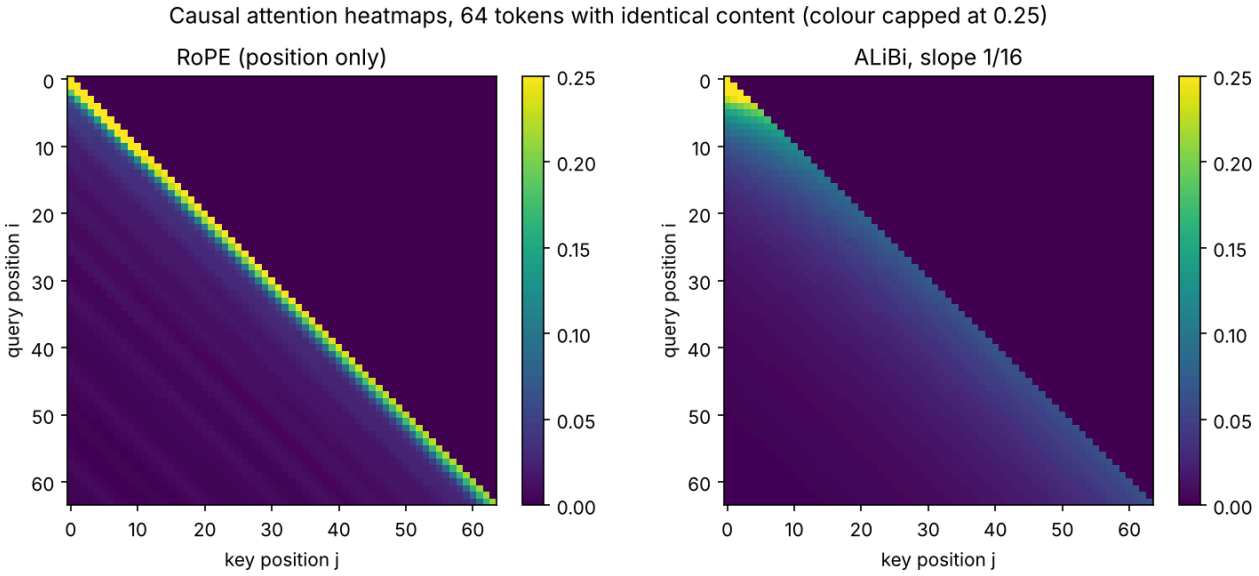
Figures written to figures/02_rope_vs_alibi_decay.png, figures/02_attention_heatmaps.png

[OK] Positional engine online.
```

Figures







Source code

```
"""
Mission 03 — Part 2: Communications & Protocol Officer
Rotary Positional Embeddings (RoPE, Su et al. 2021): complex 2D rotation, relative invariance,
RoPE vs ALiBi distance decay, and context-length extrapolation.

USSE Noosphere · Sector SEC-03-M45 (Pleiades) · Subsystem 03: Multi-Head Attention Mesh
Squadron Colby

Run:  python 02_RoPE_Positional_Engine.py
      (writes figures/02_rope_vs_alibi_decay.png and figures/02_attention_heatmaps.png)
"""
from __future__ import annotations

import math
import pathlib
from typing import Tuple

import torch

FIG_DIR = pathlib.Path(__file__).resolve().parent / "figures"


# ---------------------------------------------------------------------------- #
# 1. RoPE core
# ---------------------------------------------------------------------------- #

def rope_frequencies(dim: int, base: float = 10000.0) -> torch.Tensor:
    """theta_i = base^(-2(i-1)/d), i = 1..d/2  (one frequency per 2D plane)."""
    if dim % 2:
        raise ValueError("RoPE needs an even head dimension (dimensions are paired).")
    i = torch.arange(0, dim, 2, dtype=torch.float64)
```

```
    return base ** (-i / dim)


def precompute_freqs_cis(dim: int, max_len: int, base: float = 10000.0,
                         scale: float = 1.0) -> torch.Tensor:
    """cis(m * theta_i) = e^{i m theta_i} for every position m < max_len.

    `scale` > 1 is linear Position Interpolation (Chen et al. 2023): positions are
    divided by `scale` so a longer context reuses angles seen in training.
    Returns a complex64 tensor of shape (max_len, dim // 2).
    """
    theta = rope_frequencies(dim, base)
    m = torch.arange(max_len, dtype=torch.float64) / scale
    angles = torch.outer(m, theta)
    return torch.polar(torch.ones_like(angles), angles).to(torch.complex64)


def apply_rope(x: torch.Tensor, freqs_cis: torch.Tensor) -> torch.Tensor:
    """Rotate each consecutive pair (x_{2i}, x_{2i+1}) by angle m * theta_i.

    x:         (..., seq_len, head_dim) real; works for (B, h, N, d) query or key tensors
    freqs_cis: (seq_len, head_dim // 2) complex, rows = positions of x's tokens
    The pair is treated as one complex number z = x_{2i} + i x_{2i+1};
    multiplying by e^{i m theta} is exactly a 2D rotation matrix
        [cos -sin; sin cos] applied to (x_{2i}, x_{2i+1}).
    """
    z = torch.view_as_complex(x.float().reshape(*x.shape[:-1], -1, 2).contiguous())
    rotated = z * freqs_cis.view(*([1] * (z.dim() - 2)), *freqs_cis.shape)
    return torch.view_as_real(rotated).flatten(-2).type_as(x)


def apply_rotary_emb(xq: torch.Tensor, xk: torch.Tensor,
                     freqs_cis: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """Starter-compatible wrapper: rotate queries and keys with the same table."""
    return apply_rope(xq, freqs_cis), apply_rope(xk, freqs_cis)


def rope_score(q: torch.Tensor, k: torch.Tensor, m: int, n: int, freqs_cis: torch.Tensor) -> float:
    """<RoPE(q, m), RoPE(k, n)> for single vectors q, k of shape (d,)."""
    qm = apply_rope(q[None, :], freqs_cis[m:m + 1])[0]
    kn = apply_rope(k[None, :], freqs_cis[n:n + 1])[0]
    return float((qm.double() * kn.double()).sum())


# ---------------------------------------------------------------------------- #
# 2. Verification
# ---------------------------------------------------------------------------- #

def verify_rotation_is_orthogonal(dim: int = 64, seed: int = 1) -> float:
    """A rotation never changes length: ||RoPE(x, m)|| == ||x|| for every m."""
    g = torch.Generator().manual_seed(seed)
    x = torch.randn(200, dim, generator=g)
    rot = apply_rope(x, precompute_freqs_cis(dim, 200))
    err = (rot.norm(dim=-1) - x.norm(dim=-1)).abs().max().item()
    print(f"norm preserved under rotation: max |‖RoPE(x)‖ − ‖x‖| = {err:.2e}")
    assert err < 1e-4
    return err


def verify_relative_invariance(dim: int = 64, trials: int = 200, max_len: int = 4096,
                               seed: int = 42) -> float:
    """<RoPE(q,m), RoPE(k,n)> == <RoPE(q,m+δ), RoPE(k,n+δ)> for random q, k, m, n, δ.

    Proof sketch (one plane): z_q e^{i m θ} · conj(z_k e^{i n θ}) = z_q conj(z_k) e^{i (m−n) θ};
    the real part is the dot product, and it depends on m and n only through m − n.
    """
    g = torch.Generator().manual_seed(seed)
    fc = precompute_freqs_cis(dim, max_len)
    worst = 0.0
    for _ in range(trials):
        q, k = torch.randn(dim, generator=g), torch.randn(dim, generator=g)
        m, n = torch.randint(0, max_len // 2, (2,), generator=g).tolist()
        delta = int(torch.randint(1, max_len // 2, (1,), generator=g))
        a = rope_score(q, k, m, n, fc)
        b = rope_score(q, k, m + delta, n + delta, fc)
        worst = max(worst, abs(a - b) / max(1.0, abs(a)))
    print(f"relative invariance over {trials} random (q, k, m, n, δ): max relative error = {worst:.2e}")
    assert worst < 1e-4, "RoPE relative invariance violated"
    return worst
```

```
def verify_starter_example(dim: int = 64) -> Tuple[float, float]:
    """The instructor's fixed example: m=10, n=25, δ=40."""
    torch.manual_seed(42)
    fc = precompute_freqs_cis(dim, 256)
    q, k = torch.randn(dim), torch.randn(dim)
    a, b = rope_score(q, k, 10, 25, fc), rope_score(q, k, 50, 65, fc)
    print(f"<RoPE(q,10), RoPE(k,25)> = {a:.6f}   <RoPE(q,50), RoPE(k,65)> = {b:.6f}   "
          f"|diff| = {abs(a - b):.2e}")
    assert abs(a - b) < 1e-4
    return a, b


# ---------------------------------------------------------------------------- #
# 3. RoPE vs ALiBi distance behaviour
# ---------------------------------------------------------------------------- #

def rope_distance_curve(dim: int = 128, max_dist: int = 512, base: float = 10000.0) -> torch.Tensor:
    """Normalised score s(Δ) = <RoPE(q, Δ), RoPE(k, 0)> / <q, k> for q = k = all-ones.

    With identical content, any change with Δ is caused by position alone.
    s(Δ) = (2/d) Σ_i cos(Δ θ_i): it starts at 1 and decays (with ripples) as Δ grows,
    the "long-term decay" property in Su et al. (2021), §3.4.3.
    """
    theta = rope_frequencies(dim, base)
    d = torch.arange(max_dist + 1, dtype=torch.float64)
    return torch.cos(torch.outer(d, theta)).mean(dim=1)


def alibi_slopes(num_heads: int) -> torch.Tensor:
    """ALiBi head slopes 2^(-8h/H), h = 1..H (Press et al. 2022)."""
    h = torch.arange(1, num_heads + 1, dtype=torch.float64)
    return 2.0 ** (-8.0 * h / num_heads)


def attention_by_distance(bias_or_score: torch.Tensor) -> torch.Tensor:
    """Softmax weight a query at the end of a window gives to each key at distance Δ."""
    return torch.softmax(bias_or_score, dim=0)


def attention_heatmaps(n: int = 64, dim: int = 64, logit_scale: float = 8.0,
                       alibi_slope: float = 0.0625) -> Tuple[torch.Tensor, torch.Tensor]:
    """Causal attention matrices driven by position only (identical content at every token).

    RoPE: scores = logit_scale * s(i − j);  ALiBi: scores = −slope * (i − j).
    """
    s = rope_distance_curve(dim, n)
    i = torch.arange(n)
    dist = (i[:, None] - i[None, :]).clamp_min(0)
    future = i[None, :] > i[:, None]
    rope = (logit_scale * s[dist]).masked_fill(future, float("-inf"))
    alibi = (-alibi_slope * dist.double()).masked_fill(future, float("-inf"))
    return torch.softmax(rope, -1), torch.softmax(alibi, -1)


# ---------------------------------------------------------------------------- #
# 4. Context-length extrapolation
# ---------------------------------------------------------------------------- #

def extrapolation_report(dim: int = 128, train_len: int = 4096, target_len: int = 32768,
                         base: float = 10000.0) -> dict:
    """Which RoPE planes see new (never-trained) angles when the context grows?

    Plane i completes a full turn every 2π/θ_i tokens. Planes whose wavelength is longer
    than the training length never saw their full circle; beyond train_len they produce
    angles the model has never seen. Position Interpolation (scale = target/train) maps
    every position back inside the trained angle range.
    """
    theta = rope_frequencies(dim, base)
    wavelength = 2 * math.pi / theta
    unseen = int((wavelength > train_len).sum())
    max_angle_train = (train_len - 1) * theta[-1].item()
    max_angle_target = (target_len - 1) * theta[-1].item()
    max_angle_pi = (target_len - 1) / (target_len / train_len) * theta[-1].item()
    return {"dim": dim, "planes": dim // 2, "train_len": train_len, "target_len": target_len,
            "planes_with_incomplete_cycle": unseen,
            "slowest_plane_angle_train_rad": max_angle_train,
            "slowest_plane_angle_target_rad": max_angle_target,
            "slowest_plane_angle_with_PI_rad": max_angle_pi,
            "pi_scale": target_len / train_len}
```

```
def plot_figures() -> None:
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("(matplotlib not installed: skipping figures)")
        return
    FIG_DIR.mkdir(exist_ok=True)
    max_d = 256
    s = rope_distance_curve(128, max_d)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    ax = axes[0]
    ax.plot(s.numpy(), label="RoPE  (d=128, base 10000)", color="#1f77b4")
    for slope in alibi_slopes(8)[[0, 2, 4]]:
        ax.plot((-slope * torch.arange(max_d + 1)).numpy() / 16, "--",
                label=f"ALiBi bias / 16  (slope {slope:.3f})")
    ax.axhline(0, color="grey", lw=0.6)
    ax.set_xlabel("distance Δ = i − j (tokens)"); ax.set_ylabel("position-only score")
    ax.set_title("Score vs distance: smooth decay vs linear penalty")
    ax.legend(fontsize=8)

    ax = axes[1]
    w_rope = attention_by_distance(8.0 * s.flip(0))
    ax.plot(w_rope.flip(0).numpy(), label="RoPE (logit scale 8)", color="#1f77b4")
    for slope in alibi_slopes(8)[[2, 4]]:
        w = attention_by_distance((-slope * torch.arange(max_d + 1)).flip(0))
        ax.plot(w.flip(0).numpy(), "--", label=f"ALiBi slope {slope:.3f}")
    ax.set_xlabel("distance Δ (tokens)"); ax.set_ylabel("attention weight")
    ax.set_title("Attention weight by distance (256-token window)")
    ax.set_xlim(0, 128); ax.legend(fontsize=8)
    fig.tight_layout(); fig.savefig(FIG_DIR / "02_rope_vs_alibi_decay.png", dpi=150); plt.close(fig)

    a_rope, a_alibi = attention_heatmaps()
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.4))
    for ax, a, t in [(axes[0], a_rope, "RoPE (position only)"), (axes[1], a_alibi, "ALiBi, slope 1/16")]:
        im = ax.imshow(a.numpy(), cmap="viridis", vmin=0, vmax=0.25)
        ax.set_title(t); ax.set_xlabel("key position j"); ax.set_ylabel("query position i")
        fig.colorbar(im, ax=ax, fraction=0.046)
    fig.suptitle("Causal attention heatmaps, 64 tokens with identical content (colour capped at 0.25)")
    fig.tight_layout(); fig.savefig(FIG_DIR / "02_attention_heatmaps.png", dpi=150); plt.close(fig)
    print("Figures written to figures/02_rope_vs_alibi_decay.png, figures/02_attention_heatmaps.png\n")


def main() -> None:
    print("USSE NOOSPHERE · SUBSYSTEM 03 · PART 2: ROTARY POSITIONAL EMBEDDINGS\n")
    print("=== Verification ===")
    verify_rotation_is_orthogonal()
    verify_starter_example()
    verify_relative_invariance()
    x = torch.randn(2, 8, 16, 64)
    xq, xk = apply_rotary_emb(x, x, precompute_freqs_cis(64, 16))
    print(f"batched (B, h, N, d) = {tuple(x.shape)} -> {tuple(xq.shape)}")
    print("[PASS] RoPE is a pure rotation and the score depends only on m − n.\n")

    print("=== RoPE vs ALiBi: position-only score by distance ===")
    s = rope_distance_curve(128, 4096)
    slope = alibi_slopes(8)[2].item()
    print(f"{'Δ':>6} | {'RoPE s(Δ)':>9} | {'ALiBi bias (slope ' + f'{slope:.3f})':>22}")
    for d in (0, 1, 4, 16, 64, 256, 1024, 4096):
        print(f"{d:>6} | {s[d].item():>9.3f} | {-slope * d:>22.2f}")
    print("RoPE decays smoothly and stays bounded (|s| ≤ 1); ALiBi keeps falling linearly forever,\n"
          "so far tokens are pushed to ~0 weight: strong recency, simple extrapolation.\n")

    print("=== Context-length extrapolation (d=128, base 10000) ===")
    r = extrapolation_report()
    print(f"trained on {r['train_len']} tokens, asked for {r['target_len']}:")
    print(f"  {r['planes_with_incomplete_cycle']} of {r['planes']} planes never completed a full turn in training")
    print(f"  slowest plane angle: {r['slowest_plane_angle_train_rad']:.3f} rad at train length -> "
          f"{r['slowest_plane_angle_target_rad']:.3f} rad at target (unseen)")
    print(f"  Position Interpolation (scale {r['pi_scale']:.0f}×) brings it back to "
          f"{r['slowest_plane_angle_with_PI_rad']:.3f} rad (inside the trained range)\n")
    plot_figures()
    print("[OK] Positional engine online.")


if __name__ == "__main__":
    main()
```